SBB-Verspätungsanalyse – Dokumentation (M323)

Projektgruppe: Aimen Labidi, Elias Felder

Modul 323, Funktional programmieren

Version 2.0

Inhaltsverzeichnis

1. Gewählte Programmiersprache: Python 2
1.1 Warum Python? 2
1.2 Funktionale Sprachelemente von Python 2
1.3 Grenzen der funktionalen Programmierung in Python 5
1.4 Übersicht: Einsatz in unserem Projekt (V2.0) 5
2. Projektantrag 6
2.1 Umsetzung und Präzisierungen 7
3. Output 7
3.1 Programm starten 7
3.2 Output Version 1.0 8
3.3 Output Version 2.0 9
3.4 So liest man den Output 9
4. Fazit 10
4.1 Vergleich am Beispiel von Funktion 4 (Ausfälle pro Tag) 10
4.2 Nutzen für unser Projekt und Unterschied zur imperativen Lösung 10
4.3 Hat das Refactoring zu einer Vereinfachung geführt? 10
4.4 Würden wir funktionale Elemente wieder verwenden? 11
4.5 Anwendungsfälle im Betrieb 11
5. Quellen 11
Anhang: Genehmigter Projektantrag 12

1. Gewählte Programmiersprache: Python

1.1 Warum Python?

Python ist eine **Multi-Paradigmen-Sprache**: Sie ist im Kern imperativ und objektorientiert (Anweisungen, Schleifen, veränderbare Variablen, Klassen), bietet aber viele funktionale Sprachelemente. Deshalb lassen sich Version 1.0 (imperativ) und Version 2.0 (funktional) in derselben Sprache schreiben und direkt vergleichen.

Weitere Gründe: Wir kennen Python aus anderen Modulen, das Einlesen von CSV-Dateien (`csv`) und das Rechnen mit Datum und Zeit (`datetime`) sind in der Standardbibliothek enthalten, und es braucht keine Kompilierung und keine externen Bibliotheken.

1.2 Funktionale Sprachelemente von Python

Die Beispiele in diesem Kapitel haben wir selbst geschrieben. Die Grundlagen stammen aus der offiziellen Python-Dokumentation (siehe Kapitel 5, Quellen). Die Zeilen mit `# ->` zeigen jeweils das Resultat.

Funktionen als Werte (First-Class-Functions)

Funktionen sind in Python Objekte. Man kann sie in Variablen speichern, in Listen ablegen, anderen Funktionen übergeben und als Resultat zurückgeben.

```
def verdoppeln(x):
    return x * 2

f = verdoppeln            # Funktion in Variable speichern (ohne Aufruf)
f(21)                     # -> 42
```

Funktionen höherer Ordnung

Eine Funktion höherer Ordnung nimmt eine Funktion als Parameter entgegen oder gibt eine Funktion zurück. `map`, `filter`, `reduce` und `sorted` sind solche Funktionen; man kann aber auch eigene schreiben.

```
def zweimal(funktion, wert):
    return funktion(funktion(wert))

zweimal(verdoppeln, 5)    # -> 20
```

map und filter

`map(f, daten)` wendet `f` auf jedes Element an, `filter(f, daten)` behält nur die Elemente, für die `f` `True` liefert. Beide geben einen **Iterator** zurück, der erst rechnet, wenn die Werte gebraucht werden (Lazy Evaluation). Mit `list(...)` werden alle Werte erzeugt.

```
zahlen = [1, 2, 3, 4, 5]
list(map(lambda x: x * x, zahlen))          # -> [1, 4, 9, 16, 25]
list(filter(lambda x: x % 2 == 0, zahlen))  # -> [2, 4]
```

functools.reduce

`reduce(f, daten, start)` fasst eine Folge schrittweise zusammen: `f` bekommt das bisherige Zwischenresultat (Akkumulator) und das nächste Element. Seit Python 3 ist `reduce` keine eingebaute Funktion mehr, sondern muss aus `functools` importiert werden. Der Akkumulator muss keine Zahl sein, er kann z. B. auch ein Dict sein (so zählt V2 die Werte pro Bahnhof).

```
from functools import reduce
reduce(lambda summe, x: summe + x, [1, 2, 3, 4], 0)   # -> 10
reduce(lambda d, wort: {**d, wort: d.get(wort, 0) + 1}, ["a", "b", "a"], {})
                                                      # -> {'a': 2, 'b': 1}
```

Lambda-Ausdrücke

Mit `lambda` erstellt man kleine, anonyme Funktionen direkt dort, wo man sie braucht, z. B. als Sortierschlüssel. Ein Lambda darf nur **einen Ausdruck** enthalten, keine Anweisungen wie `if`-Blöcke, Schleifen oder Zuweisungen. Für mehr Logik schreibt man eine normale Funktion mit `def`.

```
quadrat = lambda x: x * x
quadrat(7)                                  # -> 49
```

Comprehensions

Eine List-Comprehension verbindet Map und Filter in einem Ausdruck: `[ausdruck for x in daten if bedingung]`. Es gibt sie auch für Dicts (`{k: v for ...}`) und Sets (`{x for ...}`). Mehrere `for` hintereinander durchlaufen verschachtelte Daten, das nutzt V2, um die 3D-Struktur flach zu machen.

```
[x * x for x in range(6) if x % 2 == 0]      # -> [0, 4, 16]
{wort: len(wort) for wort in ["Bern", "Olten"]}  # -> {'Bern': 4, 'Olten': 5}
[zahl for zeile in [[1, 2], [3]] for zahl in zeile]  # -> [1, 2, 3]
```

Generatoren

Ein Generator-Ausdruck sieht aus wie eine List-Comprehension, aber mit runden Klammern. Er erzeugt die Werte erst bei Bedarf und speichert keine ganze Liste. Eine Funktion mit `yield` ist eine Generator-Funktion: Sie liefert Werte nacheinander und merkt sich dazwischen, wo sie stehen geblieben ist.

```
sum(x * x for x in range(1_000_000))   # keine Liste mit 1 Mio. Elementen nötig

def countdown(n):
    while n > 0:
        yield n
        n -= 1

list(countdown(3))                     # -> [3, 2, 1]
```

Closures

Eine innere Funktion kann auf Variablen der äusseren Funktion zugreifen, auch nachdem die äussere Funktion beendet ist. So kann man Funktionen «konfigurieren».

```
def ist_verspaetet_ab(grenze):
    return lambda minuten: minuten >= grenze

ab_3 = ist_verspaetet_ab(3)
ab_3(2.5), ab_3(4)                     # -> (False, True)
```

functools.partial

`partial` erzeugt aus einer Funktion eine neue Funktion, bei der einige Parameter schon festgelegt sind (partielle Anwendung).

```
from functools import partial
binaer_zu_int = partial(int, base=2)
binaer_zu_int("1010")                  # -> 10
```

Decorators

Ein Decorator ist eine Funktion höherer Ordnung, die eine Funktion entgegennimmt und eine erweiterte Funktion zurückgibt. Mit `@` wird er auf eine Funktion angewendet. Ein bekanntes Beispiel aus der Standardbibliothek ist `functools.cache`, das Resultate einer puren Funktion zwischenspeichert.

```
from functools import cache

@cache
def fibonacci(n):
    return n if n < 2 else fibonacci(n - 1) + fibonacci(n - 2)

fibonacci(80)                          # -> 23416728348467685 (sofort, dank Cache)
```

Eingebaute Funktionen mit Funktionsparametern

`sorted`, `min` und `max` nehmen mit `key` eine Funktion entgegen, nach der verglichen wird. `sorted` gibt immer eine **neue** Liste zurück und verändert die ursprüngliche nicht. Dazu kommen `any`/`all` (gilt eine Bedingung für mindestens ein bzw. alle Elemente), `zip` (Folgen paarweise verbinden) und `enumerate` (Elemente nummerieren).

```
bahnhoefe = [("Bern", 95.9), ("Olten", 94.4), ("Basel SBB", 90.3)]
sorted(bahnhoefe, key=lambda b: b[1])         # aufsteigend nach Quote
max(bahnhoefe, key=lambda b: b[1])            # -> ('Bern', 95.9)
all(quote > 90 for _, quote in bahnhoefe)     # -> True
list(zip(["a", "b"], [1, 2]))                 # -> [('a', 1), ('b', 2)]
list(enumerate(["Bern", "Olten"], start=1))   # -> [(1, 'Bern'), (2, 'Olten')]
```

Modul operator

Es stellt die Operatoren von Python als Funktionen zur Verfügung, damit man nicht für alles ein Lambda schreiben muss. `itemgetter("bahnhof")` ist gleichbedeutend mit `lambda d: d["bahnhof"]`; V2 benutzt es als Schlüssel für `groupby`.

```
from operator import add, itemgetter
reduce(add, [1, 2, 3])                          # -> 6
itemgetter("bahnhof")({"bahnhof": "Bern"})      # -> 'Bern'
```

Modul itertools

Funktionen, die Iteratoren kombinieren. `groupby` fasst **aufeinanderfolgende** Elemente mit gleichem Schlüssel zu Gruppen zusammen (die Daten müssen also nach dem Schlüssel sortiert sein). `chain` hängt Folgen aneinander, `accumulate` liefert laufende Zwischenresultate.

```
from itertools import accumulate, chain, groupby
[(k, len(list(g))) for k, g in groupby("aaabcc")]   # -> [('a', 3), ('b', 1), ('c', 2)]
list(chain([1, 2], [3]))                            # -> [1, 2, 3]
list(accumulate([1, 2, 3, 4]))                      # -> [1, 3, 6, 10]
```

Unveränderlichkeit (Immutability)

`int`, `float`, `str`, `tuple` und `frozenset` sind unveränderlich. Mit `collections.namedtuple` oder `@dataclass(frozen=True)` kann man unveränderliche Datensätze definieren. Listen und Dicts sind veränderlich; funktional arbeitet man deshalb so, dass statt einer Änderung ein **neues** Objekt erzeugt wird, z. B. mit `{**dict, schluessel: wert}`.

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Halt:
    linie: str
    verspaetung: float

h = Halt("IC6", 3.5)
# h.verspaetung = 0      -> FrozenInstanceError

alt = {"linie": "S3"}
neu = {**alt, "verspaetung": 1.5}   # alt bleibt {'linie': 'S3'}
```

Pure Functions

Eine pure Funktion liefert für dieselben Parameter immer dasselbe Resultat und hat keine Seiteneffekte (sie verändert nichts ausserhalb, gibt nichts aus, liest keine Dateien). Python erzwingt das nicht, man muss es selbst einhalten. Pure Funktionen sind einfach zu testen und können beliebig kombiniert werden.

```
def pure_verspaetung(soll_min, ist_min):        # pur: nur Parameter, nur Rückgabe
    return max(0, ist_min - soll_min)

total = 0
def unpure_addiere(x):                          # nicht pur: verändert globale Variable
    global total
    total += x
    return total
```

Rekursion

Funktionen dürfen sich selbst aufrufen; damit lassen sich Schleifen funktional ersetzen.

```
def summe(liste):
    return 0 if not liste else liste[0] + summe(liste[1:])

summe([1, 2, 3])                        # -> 6
```

1.3 Grenzen der funktionalen Programmierung in Python

- Keine Tail-Call-Optimierung: Jeder rekursive Aufruf braucht Platz auf dem Stack. Ab ca. 1000 Aufrufen (Standardlimit) bricht Python mit `RecursionError` ab. Für grosse Datenmengen wie unsere 26 984 Halte ist Rekursion deshalb ungeeignet; wir verwenden `map`, `filter`, `reduce` und Comprehensions.
- Lambdas sind auf einen einzigen Ausdruck beschränkt.
- Unveränderlichkeit und Reinheit werden nicht erzwungen, sie sind eine Disziplin der Programmierer. Ein Lambda kann trotzdem eine Liste verändern.
- `reduce` wurde in Python 3 bewusst aus den eingebauten Funktionen entfernt, weil eine Schleife oft lesbarer ist; der Python-Stil bevorzugt Comprehensions gegenüber `map`/`filter` mit Lambda.

1.4 Übersicht: Einsatz in unserem Projekt (V2.0)

Sprachelement	Wo in V2.0
`filter` mit Prädikat-Funktion	Funktion 1: `filter(gefahren, ...)`, Funktion 5: Halte einer Kategorie
`map`	Funktion 2: `map(mit_verspaetung, ...)`, Laden: `map(zu_halt, ...)`
`functools.reduce`	Funktionen 3, 4 und 5 (Summen pro Bahnhof, Ausfälle

	pro Tag, Maximum pro Kategorie)
`lambda`	Akkumulatoren für `reduce`, Sortierschlüssel im Ranking
List-/Dict-Comprehensions, Generatoren	`alle_halte`, `ausfaelle_pro_tag`, `maxima_pro_kategorie`, `bericht`
`sorted` mit `key`	Funktion 6: Ranking
`itertools.groupby`, `operator.itemgetter`	`lade_daten`: Aufbau der 3D-Struktur
`enumerate`, `zip`	Rangnummern im Bericht, Summieren der Zähler pro Bahnhof
Unveränderlichkeit (`{**halt, ...}`, Tupel)	Keine Funktion verändert ihre Eingabedaten
Pure Functions	Alle Auswertungsfunktionen; nur `lade_daten` und `main` haben Seiteneffekte

2. Projektantrag

Der folgende Antrag wurde so von der Lehrperson genehmigt (Original im Anhang).

Projekttitel	SBB-Verspätungsanalyse – Pünktlichkeit an Schweizer Bahnhöfen
Ausgangssituation / Problembeschreibung	Jeder kennt es: Man steht am Bahnhof und der Zug kommt mal wieder zu spät. Wir wollen herausfinden, wie oft die Züge der SBB wirklich zu spät kommen, an welchen Bahnhöfen das am meisten passiert und an welchen Tagen am meisten Züge ausfallen. Die Daten dazu gibt es gratis im Internet. Die Dateien sind aber riesig und man kann sie so nicht einfach lesen. Darum schreiben wir ein Programm, das die Daten auswertet und die Resultate übersichtlich in der Konsole anzeigt.
Datengrundlage	**Quelle:** «Ist-Daten» von opentransportdata.swiss (auch auf opendata.swiss verfügbar). Pro Betriebstag gibt es eine CSV-Datei mit allen Fahrten und Halten inkl. Soll- und Ist-Zeiten sowie Ausfallkennzeichen. **Beispieldaten:** Da eine Tagesdatei sehr gross ist, erstellen wir einmalig einen reduzierten Datensatz: 7 Betriebstage, 5 Bahnhöfe (Basel SBB, Zürich HB, Bern, Olten, Liestal), nur Züge. Dieser Datensatz wird von Version 1.0 und Version 2.0 identisch verwendet. **Aufbau (3 Dimensionen):** 1. Dimension: Betriebstage (7 Tage) 2. Dimension: Bahnhöfe (5 Bahnhöfe) 3. Dimension: Halte an diesem Bahnhof an diesem Tag (Linie, Zugkategorie, Soll-Ankunft, Ist-Ankunft, Ausfall)
Produktfunktionen	1. **Filter:** Nur Halte berücksichtigen, die tatsächlich gefahren wurden (keine Ausfälle) und eine Ist-Zeit besitzen. 2. **Map:** Für jeden Halt die Verspätung in Minuten berechnen (Ist-Ankunft minus Soll-Ankunft) und ihn als «pünktlich» (unter 3 Minuten) oder «verspätet» einstufen. 3. **Reduce:** Durchschnittliche Verspätung und Pünktlichkeitsquote in Prozent pro Bahnhof berechnen. 4. **Reduce:** Anzahl Zugausfälle pro Betriebstag zählen. 5. **Filter + Reduce:** Pro Zugkategorie (IC, IR, RE, S) die maximale Verspätung inkl. Linie, Bahnhof und Datum ermitteln. 6. **Sortieren:** Ranking der Bahnhöfe vom pünktlichsten zum unpünktlichsten. *Es wird nie nur auf einen einzelnen Wert reduziert, sondern jeweils pro Bahnhof, pro Tag und pro Zugkategorie.*

Technologien	**Programmiersprache:** Python 3.12 **Frameworks:** keine, nur Standardbibliothek (csv, datetime, functools, itertools) **UI:** Output auf die Konsole mit integrierten Mitteln (print, f-Strings) **Art der Applikation:** Konsolenapplikation **Funktionale Elemente (V2.0):** lambda, map, filter, functools.reduce, List- und Dict-Comprehensions, Generatoren, sorted mit key, itertools.groupby, Funktionen höherer Ordnung
Output	Formatierter Output auf die Konsole. Version 1.0 und Version 2.0 erzeugen exakt denselben Output. Mockup: siehe Anhang.

2.1 Umsetzung und Präzisierungen

Alle sechs Produktfunktionen sind in V1.0 und V2.0 umgesetzt. Bei der Umsetzung haben wir folgende Punkte präzisiert:

- Datensatz: `data/ist_daten.csv` enthält 26 984 Halte vom 03.11.2025 bis 09.11.2025 an den fünf Bahnhöfen, Kategorien S, RE, IR und IC. Er wird mit `data/generate_data.py` aus den Tagesdateien erzeugt. Beide Versionen lesen dieselbe Datei.
- 3D-Struktur: Damit Tag und Bahnhof nicht verloren gehen, speichern wir Paare: `[(tag, [(bahnhof, [halt, ...]), ...]), ...]`. Die Soll- und Ist-Zeiten enthalten das Datum (z. B. `03.11.2025 05:14`), damit Verspätungen über Mitternacht korrekt berechnet werden.
- Zu frühe Ankünfte zählen als 0 Minuten Verspätung, sonst würden sie den Durchschnitt künstlich verkleinern.
- Ranking bei Gleichstand: zuerst Pünktlichkeitsquote absteigend, bei gleicher Quote kleinere Ø Verspätung zuerst, dann alphabetisch.
- Output: Die Zahlen im Mockup waren Platzhalter. Im Balkendiagramm steht ein # für 2 Ausfälle (sonst wäre die Zeile breiter als die Konsole), und unter dem Ranking steht zur Erklärung, ab wann ein Zug als pünktlich gilt.

3. Output

3.1 Programm starten

```
python v1/main.py      # Version 1.0 (imperativ)
python v2/main.py      # Version 2.0 (funktional)
```

3.2 Output Version 1.0

```
============================================================
   SBB Verspätungsanalyse  |  03.11.2025 - 09.11.2025
============================================================

Pünktlichkeit pro Bahnhof (Ranking)
------------------------------------------------------------
Rang  Bahnhof         Halte   Ø Verspätung  Pünktlich
  1   Bern            7'008        0.7 min     95.9 %
  2   Olten           4'128        0.8 min     94.4 %
  3   Zürich HB      11'666        1.1 min     92.4 %
  4   Liestal         1'329        1.1 min     90.9 %
  5   Basel SBB       2'226        1.0 min     90.3 %
(pünktlich = weniger als 3 Minuten Verspätung)

Zugausfälle pro Tag (# = 2 Ausfälle)
------------------------------------------------------------
Mo 03.11.   69   ##################################
Di 04.11.   39   ###################
Mi 05.11.   35   #################
Do 06.11.   47   ########################
Fr 07.11.   46   ########################
Sa 08.11.   67   #################################
So 09.11.   71   ###################################

Maximale Verspätung pro Zugkategorie
------------------------------------------------------------
IC   58 min   (IC6, Olten, 06.11.)
IR   63 min   (IR37, Zürich HB, 03.11.)
RE   17 min   (RE24, Olten, 04.11.)
S    29 min   (S12, Zürich HB, 03.11.)
============================================================
```

3.3 Output Version 2.0

```
============================================================
   SBB Verspätungsanalyse  |  03.11.2025 - 09.11.2025
============================================================

Pünktlichkeit pro Bahnhof (Ranking)
------------------------------------------------------------
Rang  Bahnhof         Halte   Ø Verspätung  Pünktlich
  1   Bern            7'008        0.7 min     95.9 %
  2   Olten           4'128        0.8 min     94.4 %
  3   Zürich HB      11'666        1.1 min     92.4 %
  4   Liestal         1'329        1.1 min     90.9 %
  5   Basel SBB       2'226        1.0 min     90.3 %
(pünktlich = weniger als 3 Minuten Verspätung)

Zugausfälle pro Tag (# = 2 Ausfälle)
------------------------------------------------------------
Mo 03.11.   69   ##################################
Di 04.11.   39   ###################
Mi 05.11.   35   #################
Do 06.11.   47   ########################
Fr 07.11.   46   ########################
Sa 08.11.   67   #################################
So 09.11.   71   ###################################

Maximale Verspätung pro Zugkategorie
------------------------------------------------------------
IC   58 min   (IC6, Olten, 06.11.)
IR   63 min   (IR37, Zürich HB, 03.11.)
RE   17 min   (RE24, Olten, 04.11.)
S    29 min   (S12, Zürich HB, 03.11.)
============================================================
```

Die beiden Outputs sind Zeichen für Zeichen identisch. Das prüft der automatische Test `tests/test_gleicher_output.py`: Er startet beide Programme und vergleicht ihren Output mit der gespeicherten Datei `tests/expected_output.txt`. Zusätzlich testet er die Funktionen 1 bis 6 beider Versionen mit kleinen, von Hand berechneten Beispielen (`python -m unittest discover tests`).

3.4 So liest man den Output

- Pünktlichkeit pro Bahnhof (Ranking): *Halte* ist die Anzahl Züge, die an diesem Bahnhof in der Woche angekommen sind (ohne Ausfälle). *Ø Verspätung* ist die durchschnittliche Verspätung bei der Ankunft. *Pünktlich* ist der Anteil der Halte mit weniger als 3 Minuten Verspätung. Rang 1 ist der pünktlichste Bahnhof. Liestal steht vor Basel SBB, obwohl die Ø Verspätung höher ist, weil das Ranking nach dem Anteil pünktlicher Züge sortiert: In Liestal kommen mehr Züge pünktlich an, aber einzelne stark verspätete Züge heben den Durchschnitt an.
- Zugausfälle pro Tag: Anzahl ausgefallener Halte an allen fünf Bahnhöfen zusammen, mit einem Balken zum schnellen Vergleich. Am meisten Ausfälle gab es am Wochenende und am Montag.
- Maximale Verspätung pro Zugkategorie: der am stärksten verspätete Halt jeder Kategorie mit Linie, Bahnhof und Datum.

Kontrolle: Die Summe der Halte (26 357) plus die Ausfälle (374) plus 253 Halte ohne Ist-Zeit ergibt genau die 26 984 Zeilen des Datensatzes.

4. Fazit

4.1 Vergleich am Beispiel von Funktion 4 (Ausfälle pro Tag)

V1 (imperativ):

```
resultat = []
for tag, bahnhoefe in tage:
    anzahl = 0
    for _, halte in bahnhoefe:
        for halt in halte:
            if halt["ausfall"]:
                anzahl += 1
    resultat.append((tag, anzahl))
```

V2 (funktional):

```
[(tag, reduce(lambda anzahl, halt: anzahl + halt["ausfall"],
              (halt for _, halte in bahnhoefe for halt in halte), 0))
 for tag, bahnhoefe in tage]
```

4.2 Nutzen für unser Projekt und Unterschied zur imperativen Lösung

In V1 braucht jede Auswertung verschachtelte Schleifen, Hilfsvariablen und Zähler, die laufend verändert werden (z. B. `anzahl += 1`, `werte[0] += 1`). Man muss den ganzen Ablauf im Kopf nachspielen, um zu verstehen, was am Schluss in einer Variable steht. In V2 beschreibt der Code, **was** berechnet wird, und liest sich wie die Produktfunktionen aus dem Antrag:

```
halte = list(map(mit_verspaetung, filter(gefahren, alle_halte(tage))))
```

Jede Funktion ist klein, pur und einzeln testbar. Weil keine Daten verändert werden, kann keine Funktion eine andere durch Seiteneffekte stören. In V1 mussten wir daran selbst denken und jeden Halt bewusst mit `dict(halt)` kopieren, bevor wir Felder ergänzen; sonst hätte z. B. `alle_halte` die 3D-Struktur verändert, die danach noch für die Ausfallzählung gebraucht wird. In V2 entsteht mit `{**halt, ...}` automatisch ein neues Dict.

4.3 Hat das Refactoring zu einer Vereinfachung geführt?

Ja, insgesamt deutlich, aber nicht an jeder Stelle.

Funktion	Codezeilen V1	Codezeilen V2
1 Filter	5	1
2 Map	12	4
3 Reduce pro Bahnhof	14	7
4 Reduce pro Tag	9	3
5 Filter + Reduce pro Kategorie	11	8
6 Sortieren	12	1
Total Funktionen 1 bis 6	**63**	**24**

(Zeilen ohne Docstrings gezählt.)

- Einfacher geworden: Das Ranking (Funktion 6) brauchte in V1 einen selbst geschriebenen Insertion Sort mit einer Vergleichsfunktion; in V2 ist es eine Zeile `sorted` mit einem Schlüssel (`-quote, durchschnitt,`

`name`). Filter und Map sind je ein Aufruf mit einer gut benannten Funktion. Das Laden der 3D-Struktur mit `groupby` kommt ohne das Prüfen von «letztem Tag» und «letztem Bahnhof» aus.

- Nicht einfacher geworden: `statistik_pro_bahnhof` ist in V2 zwar kürzer, aber schwerer zu lesen: Das `reduce` mit Lambda, `zip`, `map(sum, ...)` und einem Dict als Akkumulator muss man zweimal lesen. Hier ist die Schleife aus V1 für Einsteiger verständlicher.

4.4 Würden wir funktionale Elemente wieder verwenden?

Ja. Für Datenauswertungen (Filtern, Umrechnen, Gruppieren, Zusammenfassen, Sortieren) würden wir wieder funktional arbeiten, vor allem mit Comprehensions, `map`/`filter` mit benannten Funktionen und `sorted` mit `key`. Den grössten Gewinn bringen pure Funktionen und das Vermeiden von Mutation, weil der Code dadurch leichter zu testen ist. Für komplizierte Akkumulationen würden wir aber nicht alles in ein `reduce` mit Lambda packen, sondern eine benannte Hilfsfunktion schreiben oder bewusst eine kurze Schleife verwenden.

4.5 Anwendungsfälle im Betrieb

- Log-Dateien und Exporte auswerten: z. B. Fehler aus Server-Logs filtern, nach Datum gruppieren und zählen. Das ist genau das Muster Filter, Map, Reduce aus diesem Projekt.
- Datenbereinigung vor einem Import: CSV- oder Excel-Exporte aus Fachanwendungen mit kleinen, puren Funktionen prüfen und umwandeln, bevor sie in eine Datenbank geschrieben werden. Ungültige Zeilen filtern, Formate (Datum, Telefonnummern) mit `map` vereinheitlichen.
- API-Antworten (JSON) umbauen: Listen von Objekten aus einer REST-API in die Struktur bringen, die das Frontend oder ein Bericht braucht (List- und Dict-Comprehensions).
- Validierungsregeln: Regeln als kleine Funktionen schreiben und mit `all(regel(eingabe) for regel in regeln)` kombinieren; neue Regeln lassen sich hinzufügen, ohne bestehenden Code zu ändern.
- Berichte und Statistiken: z. B. Auswertungen von Tickets, Zeiterfassungen oder Inventardaten pro Abteilung, Monat oder Kategorie.

5. Quellen

- Python Software Foundation: *Functional Programming HOWTO*. https://docs.python.org/3/howto/functional.html
- Python Software Foundation: *Built-in Functions* (`map`, `filter`, `sorted`, `zip`, `enumerate`, `any`, `all`). https://docs.python.org/3/library/functions.html
- Python Software Foundation: *functools* (`reduce`, `partial`, `cache`). https://docs.python.org/3/library/functools.html
- Python Software Foundation: *itertools*. https://docs.python.org/3/library/itertools.html
- Python Software Foundation: *operator*. https://docs.python.org/3/library/operator.html
- Python Software Foundation: *Expressions* (Lambdas, Comprehensions, Generator expressions). https://docs.python.org/3/reference/expressions.html
- Python Software Foundation: *dataclasses*. https://docs.python.org/3/library/dataclasses.html
- SBB / opentransportdata.swiss: *Ist-Daten* (Datengrundlage). https://opentransportdata.swiss/de/dataset/istdaten

Anhang: Genehmigter Projektantrag

Projekttitel:	**SBB-Verspätungsanalyse – Pünktlichkeit an Schweizer Bahnhöfen**
Ausgangssituation / Problembeschreibung:	Jeder kennt es: Man steht am Bahnhof und der Zug kommt mal wieder zu spät. Wir wollen herausfinden, wie oft die Züge der SBB wirklich zu spät kommen, an welchen Bahnhöfen das am meisten passiert und an welchen Tagen am meisten Züge ausfallen. Die Daten dazu gibt es gratis im Internet. Die Dateien sind aber riesig und man kann sie so nicht einfach lesen. Darum schreiben wir ein Programm, das die Daten auswertet und die Resultate übersichtlich in der Konsole anzeigt.

Datengrundlage:	**Quelle:** «Ist-Daten» von opentransportdata.swiss (auch auf opendata.swiss verfügbar). Pro Betriebstag gibt es eine CSV-Datei mit allen Fahrten und Halten inkl. Soll- und Ist-Zeiten sowie Ausfallkennzeichen.
	Beispieldaten: Da eine Tagesdatei sehr gross ist, erstellen wir einmalig einen reduzierten Datensatz: 7 Betriebstage, 5 Bahnhöfe (Basel SBB, Zürich HB, Bern, Olten, Liestal), nur Züge. Dieser Datensatz wird von Version 1.0 und Version 2.0 identisch verwendet.
	Aufbau (3 Dimensionen):
	1. Dimension: Betriebstage (7 Tage)
	2. Dimension: Bahnhöfe (5 Bahnhöfe)
	3. Dimension: Halte an diesem Bahnhof an diesem Tag (Linie, Zugkategorie, Soll-Ankunft, Ist-Ankunft, Ausfall)
	`[ # 1. Dimension: Tage`
	`  [ # 2. Dimension: Bahnhöfe`
	`  [ # 3. Dimension: Halte`
	`  {"linie": "IC6", "kategorie": "IC", "soll": "07:32",`
	`  "ist": "07:35", "ausfall": False},`
	`  {"linie": "S3", "kategorie": "S", "soll": "07:40",`
	`  "ist": "07:40", "ausfall": False},`
	`  ...`
	`  ],`
	`  ...`
	`  ],`
	`  ...`
	`]`

Produktfunktionen:	Das Programm wendet folgende Funktionen auf die Daten an: 1. **Filter:** Nur Halte berücksichtigen, die tatsächlich gefahren wurden (keine Ausfälle) und eine Ist-Zeit besitzen. 2. **Map:** Für jeden Halt die Verspätung in Minuten berechnen (Ist-Ankunft minus Soll-Ankunft) und ihn als «pünktlich» (unter 3 Minuten) oder «verspätet» einstufen. 3. **Reduce:** Durchschnittliche Verspätung und Pünktlichkeitsquote in Prozent pro Bahnhof berechnen. 4. **Reduce:** Anzahl Zugausfälle pro Betriebstag zählen. 5. **Filter + Reduce:** Pro Zugkategorie (IC, IR, RE, S) die maximale Verspätung inkl. Linie, Bahnhof und Datum ermitteln. 6. **Sortieren:** Ranking der Bahnhöfe vom pünktlichsten zum unpünktlichsten. *Es wird nie nur auf einen einzelnen Wert reduziert, sondern jeweils pro Bahnhof, pro Tag und pro Zugkategorie.*
Technologien:	**Programmiersprache:** Python 3.12 **Frameworks:** keine – nur Standardbibliothek (csv, datetime, functools, itertools) **UI:** Output auf die Konsole mit integrierten Mitteln (print, f-Strings)

	Art der Applikation: Konsolenapplikation **Funktionale Elemente (V2.0):** lambda, map, filter, functools.reduce, List- und Dict-Comprehensions, Generatoren, sorted mit key, itertools.groupby, Funktionen höherer Ordnung

Output:	Formatierter Output auf die Konsole. Version 1.0 und Version 2.0 erzeugen exakt denselben Output.
	Mockup:
	`============================================================`
	`SBB Verspätungsanalyse | 03.11.2025 - 09.11.2025`
	`============================================================`
	
	`Pünktlichkeit pro Bahnhof (Ranking)`
	`------------------------------------------------------------`
	`Rang Bahnhof Halte Ø Verspätung Pünktlich`
	`1 Liestal 1'204 0.8 min 94.1 %`
	`2 Olten 2'876 1.3 min 89.7 %`
	`3 Bern 3'412 1.6 min 87.2 %`
	`4 Basel SBB 3'105 1.9 min 85.5 %`
	`5 Zürich HB 4'980 2.2 min 83.0 %`
	
	`Zugausfälle pro Tag`
	`------------------------------------------------------------`
	`Mo 03.11. 12 ############`
	`Di 04.11. 4 ####`
	`Mi 05.11. 7 #######`
	`...`
	
	`Maximale Verspätung pro Zugkategorie`
	`------------------------------------------------------------`
	`IC 27 min (IC1, Bern, 05.11.)`
	`IR 18 min (IR36, Basel SBB, 03.11.)`
	`RE 11 min (RE, Olten, 07.11.)`
	`S 9 min (S3, Liestal, 04.11.)`
	`============================================================`

Projektgruppe	Aimen Labidi Elias Felder

Unterschrift / Abnahme	Lehrperson Dieter Kopp _____	Teammitglieder Aimen Labidi / Elias Felder _____